

CareerPilot — Rebuild & Redesign Brief

A ready-to-paste prompt/spec for an AI coding agent (Antigravity, Copilot, etc.) to fix functionality and redesign the UI/UX

How to use this document

- Paste this whole document (or section by section) into Antigravity / Copilot / Claude Code as the task brief.
- Tell the agent to first read the existing repo (nithu0602/CareerPilot-OS) end to end before changing anything, and to list what already exists vs. what's missing against the spec below.
- Work one step of the pipeline at a time (Section 3) — don't ask for all 6 steps in one shot, or the agent will produce shallow stubs for each.
- The 'Known likely break points' checklist (Section 2) is where to start the debugging pass before any redesign work begins.

1. Product summary

CareerPilot is an autonomous, multi-step career copilot. A user uploads a resume once and moves through a guided pipeline — resume scoring, job discovery, fit analysis, tailoring, interview prep, and recruiter outreach — with each step building on the last. The product should feel like one continuous guided flow (a wizard), not six disconnected tools.

Current problems to solve:

- Functionality: parts of the pipeline (notably the Anakin-related agent step) are not working reliably.
- Design: the UI reads as a student/hackathon project rather than a polished product — inconsistent spacing, default component styling, no clear visual hierarchy or brand identity.

2. Diagnostic pass — do this before redesigning

Ask the agent to actually run the app locally, exercise every step of the flow, and capture console/network errors rather than guessing. Have it check each of these against the real repo:

1. Environment/config: are all required API keys (LLM provider, job-board APIs, scraping service) present in .env / config, and does the app fail silently or loudly when one is missing?
2. The 'Anakin' step specifically: identify which module this refers to in the repo (agent orchestration layer, job-matching agent, etc.), reproduce the failure, and paste the exact error/stack trace before attempting a fix.
3. API/rate-limit failures: job-scraping and LinkedIn-lookup steps depend on third-party sites that change markup or block automated access often — confirm whether failures are code bugs vs. upstream blocking.
4. State handoff between steps: since this is a multi-step wizard, confirm resume data, ATS score, and selected job actually persist and pass correctly from step to step (this is the most common break point in wizard-style apps).
5. Async/loading states: long-running steps (scraping, LLM calls) need to actually resolve and update the UI — check for unhandled promise rejections or missing loading/error states that make the app look 'stuck'.

3. Full pipeline specification

Six sequential steps, each its own screen, with a persistent progress indicator (stepper) across the top so the user always knows where they are and can go back. Resume + profile data entered in Step 1 should be available to every later step without re-entry.

Step 1 — Resume Analyzer (ATS Score)

Input	User uploads resume (PDF/DOCX)
Processing	• Parse resume into structured sections (contact, summary, skills, experience, education) • Score against ATS-readability heuristics (formatting, keyword density, section structure, file type) and, if a target role/JD is available, keyword match against it
Output	• Overall ATS score (0–100) shown as a prominent visual score, not just a number • Breakdown by category (formatting, keywords, structure, length)
UI notes	• Score should animate/reveal, not just appear — this is the user's first impression of the product • Show the resume preview alongside the score, not on a separate screen
Exit condition	'Next' enabled once scoring completes; disabled with a clear reason while processing or on failure

Step 2 — Improvement Suggestions

Input	Output of Step 1 (score breakdown + parsed resume)
Processing	LLM generates specific, actionable suggestions tied to the categories that scored low (not generic advice)
Output	• Prioritized list of suggestions, grouped by category • Each suggestion should reference the specific resume line/section it applies to where possible
UI notes	Checklist format so the user can mark items as addressed; show projected score impact per suggestion if feasible

Step 3 — Live Job Discovery

Input	Parsed resume/skills profile from Step 1
Processing	Job-scraper agent pulls live listings matching the resume's role/skills
Required filters	• Location (remote/hybrid/onsite + city/region) • Experience level (entry/mid/senior) • Job type (full-time/internship/contract) • Date posted / recency • Salary range (where the source provides it) • Free-text keyword field — user can type specific keywords/titles to narrow results, combined with (not replacing) the resume-based match
Output	Ranked list of job cards: title, company, location, match %, posted date, source link
UI notes	• Filter panel should be persistent (sidebar on desktop, collapsible sheet on mobile), with an active-filter-count badge • Empty state and 'no results, try broadening filters' state are required — don't let this look broken when scraping returns nothing

Step 4 — Fit Analysis

Input	Selected job (from Step 3) + resume profile
Processing	Compare job requirements against resume content to surface gaps
Output	• What's missing: skills, experience, keywords, or qualifications the job wants that the resume doesn't show • What's already a strong match, for balance — don't only show deficits
UI notes	Side-by-side or overlay comparison (job requirements vs. resume) reads clearer than a plain bullet list

Step 5 — Tailored Resume, Cover Letter & Mock Interview

Input	Selected job + fit-gap analysis
Processing	• Generate a tailored version of the resume for this specific job • Generate a matching cover letter • Generate a set of mock interview questions (mix of role-specific and behavioral) based on the job + resume
Output	• Downloadable/editable tailored resume and cover letter (not just a text blob — preserve formatting) • Interview question list, ideally with an option to answer and get feedback, at minimum a clean printable/practice view
UI notes	This screen does three distinct jobs (resume, letter, interview prep) — use tabs or sub-steps within Step 5 rather than one long scroll

Step 6 — Recruiter Outreach

Input	Target company (from the selected job)
Processing	Find recruiters/hiring-team members at that company; produce a link straight to their profile so the user can send a connection/referral request
Output	List of recruiter names/titles at the company with a direct profile link per person
UI notes	Card list with name, title, and a clear 'View profile' action per recruiter; include a short suggested outreach-message template the user can copy
Important flag for Step 6 — read before building • Automated scraping of LinkedIn profile data violates LinkedIn's Terms of Service and has resulted in legal action and account/IP bans against other companies that tried it (e.g. the long-running hiQ Labs v. LinkedIn case, plus LinkedIn's repeated enforcement against scraping tools). • Recommend building this step on LinkedIn's official APIs / Sales Navigator or a licensed data provider instead of a scraper, or scoping it	

to data the user already has access to (their own connections, manually pasted profile URLs).

- At minimum, have the agent design this step so it's not a hard dependency for the rest of the demo to work — the pipeline should degrade gracefully (e.g. 'connect your LinkedIn' or 'paste a company URL') if live scraping isn't viable.

4. UI/UX redesign requirements

The core complaint is that this looks amateur. That's almost always a symptom of inconsistency, not a lack of features. Direct the agent to apply a real design system rather than styling each screen ad hoc.

4.1 Design system, not per-page styling

- Pick one component library (e.g. shadcn/ui, Radix, or MUI) and use it everywhere — no mixing raw HTML buttons on one page and styled-components on another.
- Define design tokens once: a small palette (1 primary, 1 accent, neutrals, 1 success/1 warning/1 error), a type scale (e.g. 4–5 font sizes total), and a spacing scale (multiples of 4px) — then use only those values across every screen.
- One font family for the whole product (a clean sans-serif such as Inter, Manrope, or the system font stack); at most one secondary/display font if needed for headings.

4.2 Layout & navigation

- Persistent step indicator (stepper) at the top of every pipeline screen, showing all 6 steps, current position, and allowing back-navigation to completed steps.
- Consistent page shell: same header, same content max-width, same padding on every screen — nothing should visually 'jump' between steps.
- Responsive from the start — most users will demo this on a laptop, but it should not break on a narrower window.

4.3 States that are currently probably missing

- Loading states for every async action (skeleton screens or spinners with a label — 'Scanning live job boards...' reads far better than a bare spinner).
- Empty states for every list (no jobs found, no recruiters found, no suggestions) with a next action, not a blank screen.
- Error states that explain what happened and offer a retry, instead of the app appearing frozen.
- Success/completion feedback at the end of each step (e.g. a brief confirmation before advancing).

4.4 Visual polish pass

- Replace default browser/library styling on buttons, inputs, and cards with the chosen design system's components, consistently sized and spaced.
- Add subtle motion only where it clarifies state change (step transitions, score reveal, list loading) — not decorative animation everywhere.
- Score and match-percentage displays should be genuine data visualizations (radial/progress rings, bar breakdowns) rather than plain numbers in text.
- Establish a short product name/wordmark treatment and use it consistently in the header — this alone measurably reduces the 'hackathon project' feel.

5. Suggested build order for the agent

6. Run the diagnostic pass (Section 2) and report findings before changing UI.

7. Fix the broken pipeline step(s) so the full 6-step flow completes end-to-end with real (or clearly-mocked) data.
8. Introduce the shared design system/tokens and the persistent stepper shell — apply to Step 1 only as a proof of concept.
9. Roll the same shell/components out to Steps 2–6 one at a time, adding the missing loading/empty/error states as you go.
10. Final pass: consistency review across all 6 screens (spacing, type, color) plus a check that data actually persists correctly step to step.

6. Acceptance checklist

- A user can go from resume upload to a recruiter's profile link without hitting a dead end or unexplained error.
- Every async step shows a loading state and either a result or a clear error — never a silent blank screen.
- Every screen shares the same header, spacing, type, and component styling.
- The step indicator accurately reflects progress and supports going back.
- Filters and keyword search on Step 3 actually narrow the visible job list.
- Step 5 clearly separates tailored resume, cover letter, and mock interview questions.
- Step 6 either uses a compliant data source or degrades gracefully with a manual-input fallback.